

Python for Bioinformatics: Annotate a Data Table with Gene Descriptions

This tutorial shows how to combine two text files—one with gene annotations and one with numerical data—into a single, annotated output file using a Python script. It assumes you already have a working Python environment or IDE.

You will: 1) Understand the data format and problem. 2) Run and verify a provided script (`annotate_data.py`). 3) Learn how the script works step by step. 4) Extend it with basic sanity checks and small improvements.

1) Problem Overview

You have two files:

- **annotation.txt** – Contains *accession codes* and *descriptions* (tab-separated).
- **data.txt** – Contains the *same accession codes* (not necessarily in the same order) followed by numerical data.

You want to merge them into a single table:

Accession	Data_Columns...	Description
-----------	-----------------	-------------

That's a classic “join by key” operation, where *accession* acts as the unique identifier.

2) Example Input Files

annotation.txt

RefSeq	DNA	ID	Description
NM_000001			Example gene A
NM_000002			Example gene B

data.txt

NM_000001	10	20	30
NM_000002	40	50	60

Expected Output

Description	10	20	30
Example gene B	40	50	60

Each row from `data.txt` gets replaced by its annotation, creating an annotated dataset.

3) Running the Script

The provided script is `annotate_data.py`. It takes two input files: the annotation file and the data file.

You can run it directly inside your IDE's terminal, or with command-line arguments:

```
python3 annotate_data.py annotation.txt data.txt > annotated.tsv
```

The script prints the combined output to the terminal by default. The redirection symbol (`>`) saves it to a file called `annotated.tsv`.

To inspect the results, open `annotated.tsv` in your IDE, a text editor, or a spreadsheet viewer.

4) What Happens in the Script

Let's unpack the core logic of `annotate_data.py`.

Step 1: Read Command-Line Arguments

```
import sys
if len(sys.argv) != 3:
    sys.exit("Usage: python annotate_data.py annotation.txt data.txt")
annots_file = sys.argv[1]
data_file = sys.argv[2]
```

This ensures the user provides both filenames; otherwise, the script stops with an informative message.

Step 2: Read and Store Annotations

```
annotation_dict = {}
with open(annots_file) as f:
    header = f.readline() # skip header line
    for line in f:
        parts = line.strip().split("\t")
        if len(parts) < 2:
            continue
        accession = parts[0]
```

```
desc = "\t".join(parts[1:])
annotation_dict[accession] = desc
```

This: - Skips the first header line. - Splits each remaining line into accession and description. - Stores them in a dictionary (`annotation_dict`) for quick lookup.

Step 3: Process the Data File

```
with open(data_file) as f:
    for line in f:
        line = line.strip()
        if not line:
            break
        parts = line.split()
        accession = parts[0]
        data_values = parts[1:]
        if accession in annotation_dict:
            annotated_line = annotation_dict[accession] + "\t" + "\t".join(data_values)
            print(annotated_line)
        else:
            print(accession + "\t" + "\t".join(data_values))
```

Here: - Each non-empty line is split by whitespace. - The first field (accession) is looked up in `annotation_dict`. - If found, its description replaces the accession. - Otherwise, it prints the row unchanged.

Step 4: Stop on Blank Line

The `if not line: break` check means the script stops reading at the first blank line.

If your data file has trailing empty lines, they won't affect the output.

5) Sanity Checks and Quality Control

You should always validate your merged output. A few checks:

1. **Matching accessions** – Make sure every accession in `data.txt` exists in `annotation.txt`.
2. **Missing descriptions** – If any rows still start with an accession, they didn't find a match.
3. **Correct column separation** – Verify tabs are consistent in the final file.
4. **No extra blank lines** – Blank lines in `data.txt` can signal end-of-file early.

6) Testing with Small Example Files

Before running on large datasets, always test the logic with small files.

Example:

mini_annotation.txt

```
RefSeq DNA ID    Description
NM_000001    Example gene A
NM_000002    Example gene B
```

mini_data.txt

```
NM_000001 10 20 30
NM_000002 40 50 60
```

Run:

```
python3 annotate_data.py mini_annotation.txt mini_data.txt
```

Expected output:

```
Description 10 20 30
Example gene B 40 50 60
```

If this works, the same logic will work for your full dataset.

7) Improving the Script (Optional)

Here are some small improvements you could make to strengthen the code:

a) Count Missing Annotations

```
missing = 0
for line in open(data_file):
    parts = line.strip().split()
    if not parts:
        continue
    accession = parts[0]
    if accession not in annotation_dict:
        missing += 1
print(f"Missing {missing} annotations", file=sys.stderr)
```

b) Skip Badly Formatted Lines

Check that each line in `annotation.txt` and `data.txt` has the right number of columns before using it.

c) Handle Multiple Annotation Columns

If your annotation file includes multiple columns (e.g., gene name, function, chromosome), the script can concatenate them all together with tabs.

8) A More Robust Version (for Later Practice)

Here's a slightly safer version that uses Python's built-in `csv` module. It handles tabs and skips malformed lines gracefully.

```
#!/usr/bin/env python3
import sys, csv

if len(sys.argv) != 3:
    sys.exit(f"Usage: python3 {sys.argv[0]} <annotations.tsv> <data.txt>")

annots_file, data_file = sys.argv[1], sys.argv[2]

# Read annotation file
annotations = {}
with open(annots_file, newline='') as fa:
    reader = csv.reader(fa, delimiter='\t')
    next(reader) # skip header
    for row in reader:
        if len(row) < 2:
            continue
        annotations[row[0]] = "\t".join(row[1:])

# Combine data and annotation
with open(data_file) as fd:
    for line in fd:
        line = line.strip()
        if not line:
            continue
        parts = line.split()
        acc = parts[0]
        data = "\t".join(parts[1:])
        if acc in annotations:
            print(f"{annotations[acc]}\t{data}")
        else:
            print(f"{acc}\t{data}")
```

You can try replacing your current script with this one if you'd like cleaner error handling.

9) Key Concepts You Learned

- **Dictionary lookups** – Storing and retrieving data by key.
 - **File I/O** – Reading text files line by line.
 - **String splitting and joining** – Using `split()` and `join()` to parse and format data.
 - **Command-line arguments** – Accessing file names from `sys.argv`.
 - **Error handling** – Stopping gracefully if inputs are missing or malformed.
-

10) Next Steps

- Try merging larger datasets.
- Experiment with `pandas` for the same task (`merge()` function).
- Add more metadata columns from your annotation file.
- Write results directly to a new file instead of redirecting output.

This kind of “join by key” is fundamental in bioinformatics data preprocessing—whether for gene annotation, sample metadata integration, or differential expression result labeling.